MIPS SIMULATOR

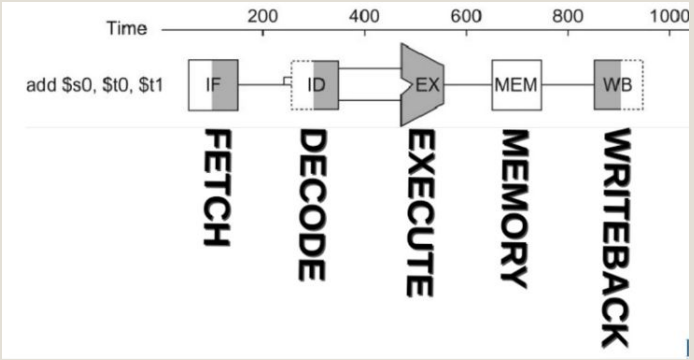
Zach Mccall, Ben Nguyen, Cole Compton, Lester Hadley

Overview

- Simulate a 5-stage MIPS pipeline
  - Written in C++
- Mips.h
  - Declares functions
- Mips.cpp
  - Pipeline to execute Instructions
- Parser.cpp
  - Reads and formats instructions for Mips.cpp
- Main.cpp
  - File that executes the mips sim
- How we got started:

```
struct Instruction {
    std::string opcode;
    int rs        = 0;
    int rt        = 0;
    int rd        = 0;
    int immediate = 0;
    int shamt     = 0;
    int address   = 0;
};

struct ControlSignals {
    bool RegWrite = false;
    bool MemRead  = false;
    bool MemWrite = false;
    bool MemToReg = false;
    bool ALUSrc   = false;
    bool Branch   = false;
    bool Jump     = false;
    int  ALUOp    = 0;
};
```



System Architecture

Components

- Register file
- Memory
- ALU
- Control unit
- Pipeline registers

```
// ── ALU ──────────────────────────────────────────────────────────
// ALUOp: 0=ADD, 1=SUB, 2=MUL, 3=AND, 4=OR, 5=SLL, 6=SRL

int alu(int a, int b, int aluOp) {
    switch (aluOp) {
        case 0: return a + b;
        case 1: return a - b;
        case 2: return a * b;
        case 3: return a & b;
        case 4: return a | b;
        case 5: return b << a;
        case 6: return (int)((unsigned)b >> a);
        default: return 0;
    }
}
```

```
void decode() {
    if (if_id.instr.opcode.empty()) { id_ex = ID_EX(); return; }

    id_ex.instr     = if_id.instr;
    id_ex.pc        = if_id.pc;
    id_ex.ctrl      = decodeControl(if_id.instr.opcode);
    id_ex.readData1 = registers[if_id.instr.rs];
    id_ex.readData2 = registers[if_id.instr.rt];
}
```

5 stage pipeline

1. IF - Instruction Fetch
2. ID - Instruction Decode
3. EX - Execute
4. MEM - Memory
5. WB - Write back

Purpose: Act as buffers between pipeline stages

- **Store instruction data and control signals**
- **Prevent data from being overwritten between cycles**

IF/ID → Stores fetched instruction + PC
ID/EX → Stores register values + control signals
EX/MEM → Stores ALU result + memory info
MEM/WB → Stores final result for write-back

System Architecture

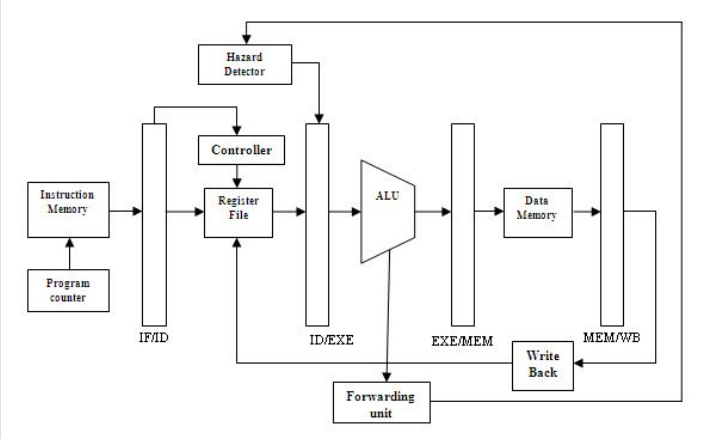
Register File

Purpose : Provides high-speed access to operands.

- **The Physical Structure**
  - An high-speed storage array of 32 registers
  - Each register is 32 bits wide
  - Register 0 ($0) - This register is always 0

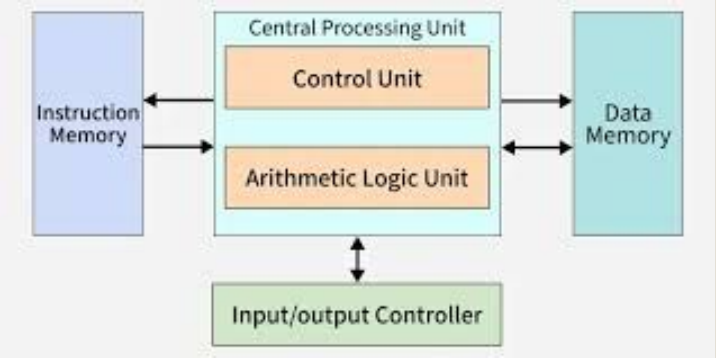
- **Ports and Interface**
  - Reads two operands at the same time during the ID stage
  - Writes one result to a register during the WB stage

- **The Half- Cycle**
  - A MIPS designs write to the register file during the first half of the clock cycle and read during the second half.
  - This allows an instruction to read a value that was just written immediately.



Memory

Purpose : Stores the Code and Data for the Program.



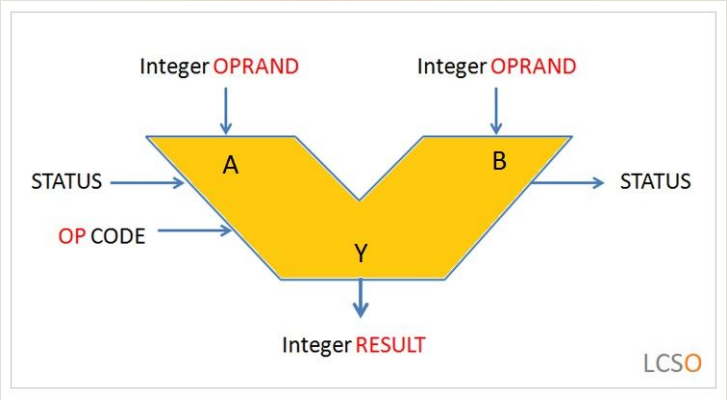
- **Divided into two stages Instruction Memory and Data Memory**
- **Instruction Memory**
  - A read-only space where data for the machine code stays.
  - The PC is a pointer in this space.
- **Data Memory**
  - Input: Takes the address from the ALU and the data from the register file.
  - Control Signal: MemRead, activated for LW, tells the memory to put data at the address to the bus.
  - Control Signal: MemWrite, activated by SW, tells the memory to take the value from the bus and store the address.
  - Output: For a LW instruction, reads data from memory is passed into MEM/WB pipeline register.

ALU

Purpose : Performs Arithmetic and Logical Operations

- **Inputs**
  - Input A, most of the time, comes from the Register File. (rs register)
  - Input B, can be the second register value (rt) or R-type instructions (ADD or SUB)
    - A sign-extended Immediate value for I-type instructions (ADDI or lw)

- **Branching**
  - When you execute BEQ instruction, it will subtract the inputs
  - If the result is 0 the zero bit is set to (1) (true)




```
// — ALU ————————————————————————————————————————————————————————
// ALUOp: 0=ADD, 1=SUB, 2=MUL, 3=AND, 4=OR, 5=SLL, 6=SRL

int alu(int a, int b, int aluOp) {
    switch (aluOp) {
        case 0: return a + b;
        case 1: return a - b;
        case 2: return a * b;
        case 3: return a & b;
        case 4: return a | b;
        case 5: return b << a;
        case 6: return (int)((unsigned)b >> a);
        default: return 0;
    }
}
```

Control Unit

Purpose : Decodes the Opcode of the Instruction.

- **Instruction Recognition**
  - Determines what type of instruction is, R-type (ADD), I-type (LW), J-type (JUMP)

- **Command Signals**
  - **RegWrite**
    - Tells the register file if it should save the incoming data
  - **ALUSrc**
    - Decides if the ALU second input is a register value or a Sign-extended immediate number.
  - **MemRead/MemWrite**
    - Allows the Data Mem or output to the bus or accept data to be stored
  - **MemtoReg**
    - Chooses whether the data being sent to the register file comes from ALU or MEM
  - **RegDst**
    - Determines which instruction field (rd or rt) is treated as the destination register address

```cpp
//Control Unit

ControlSignals decodeControl(const std::string& opcode) {
    ControlSignals ctrl;

    if (opcode == "ADD") {
        ctrl.RegWrite = true; ctrl.ALUOp = 0;
    } else if (opcode == "ADDI") {
        ctrl.RegWrite = true; ctrl.ALUSrc = true; ctrl.ALUOp = 0;
    } else if (opcode == "SUB") {
        ctrl.RegWrite = true; ctrl.ALUOp = 1;
    } else if (opcode == "MUL") {
        ctrl.RegWrite = true; ctrl.ALUOp = 2;
    } else if (opcode == "AND") {
        ctrl.RegWrite = true; ctrl.ALUOp = 3;
    } else if (opcode == "OR") {
        ctrl.RegWrite = true; ctrl.ALUOp = 4;
    } else if (opcode == "SLL") {
        ctrl.RegWrite = true; ctrl.ALUSrc = true; ctrl.ALUOp = 5;
    } else if (opcode == "SRL") {
        ctrl.RegWrite = true; ctrl.ALUSrc = true; ctrl.ALUOp = 6;
    } else if (opcode == "LW") {
        ctrl.RegWrite = true; ctrl.MemRead = true;
        ctrl.MemToReg = true; ctrl.ALUSrc  = true; ctrl.ALUOp = 0;
    } else if (opcode == "SW") {
        ctrl.MemWrite = true; ctrl.ALUSrc = true; ctrl.ALUOp = 0;
    } else if (opcode == "BEQ") {
        ctrl.Branch = true; ctrl.ALUOp = 1;
    } else if (opcode == "J") {
        ctrl.Jump = true;
    }
    // NOP: all signals remain false/0

    return ctrl;
}
```

Execution

- Parsed into structured format
  - Instructions split into fields:
    - Opcode (operation type)
    - Source registers
    - Destination registers
- Controls signals generated based on instruction type
  - **Control unit set based on opcode**
    - (RegWrite, ALUsrc. MemRead, etc…)
- Instructions pass through pipeline stages
  - IF → ID → EX → MEM → WB
- Each cycle updates all pipeline stages

```
#include "mips.h"
#include "parser.h"

int main(int argc, char* argv[]) {
    if (argc < 2) {
        std::cout << "Usage: mips_sim <program.asm> [--debug]\n";
        return 1;
    }

    for (int i = 2; i < argc; i++)
        if (std::string(argv[i]) == "--debug") debugMode = true;

    std::vector<Instruction> program = parseProgram(argv[1]);

    int cycle = 0;
    while (PC < (int)program.size() ||
           !if_id.instr.opcode.empty()  ||
           !id_ex.instr.opcode.empty()  ||
           !ex_mem.instr.opcode.empty() ||
           !mem_wb.instr.opcode.empty()) {

        writeBack();
        memAccess();
        execute();
        decode();
        fetch(program);
        cycle++;

        if (debugMode) printState(cycle);
    }

    printFinal();
    return 0;
}
```

5 Stage Pipeline

Instruction Fetch

- The instruction fetch takes a vector filled with opcode instructions acquired from the parser.
- Grabs the next instruction from the program list using PC as an index and stores it in the IF/ID
- Increments PC by 1 to point to the next instruction
- If PC is past the end of the program, it clears the IF/ID with an empty instruction to signal the pipeline there is nothing left to fetch

```
void fetch(const std::vector<Instruction>& program) {
    if (PC < (int)program.size()) {
        if_id.instr = program[PC];
        if_id.pc    = PC;
        PC++;
    } else {
        if_id = IF_ID();
    }
}
```

Instruction Decode

```
void decode() {
    if (if_id.instr.opcode.empty()) { id_ex = ID_EX(); return; }

    id_ex.instr     = if_id.instr;
    id_ex.pc        = if_id.pc;
    id_ex.ctrl      = decodeControl(if_id.instr.opcode);
    id_ex.readData1 = registers[if_id.instr.rs];
    id_ex.readData2 = registers[if_id.instr.rt];
}
```

-First checks if the ID_EX register is empty so that the correct value can return when we print the register value. (used in the Debug Mode)

-Decodes the OpCode and gives its value to the control unit

-Moves the instruction, program counter and register values to the new pipeline register. ID_EX”

Execute

- First switches all of the instruction, Control unit values to the new EX_MEM
- Enters Read Data values into the ALU so that the ALU can execute.
- Checks what opcode instruction the ALU will be executing.
- Stores the new values for the registers into their new EX_MEM registers depending on what instruction the ALU executes.

```
void execute() {
    if (id_ex.instr.opcode.empty()) { ex_mem = EX_MEM(); return; }

    ex_mem.instr     = id_ex.instr;
    ex_mem.ctrl      = id_ex.ctrl;
    ex_mem.writeData = id_ex.readData2;

    int aluA = id_ex.readData1;
    int aluB = id_ex.ctrl.ALUSrc ? id_ex.instr.immediate : id_ex.readData2;

    if (id_ex.instr.opcode == "SLL" || id_ex.instr.opcode == "SRL") {
        aluA = id_ex.instr.shamt;
        aluB = id_ex.readData2;
    }

    ex_mem.aluResult = alu(aluA, aluB, id_ex.ctrl.ALUOp);

    if (id_ex.instr.opcode == "ADD" || id_ex.instr.opcode == "SUB" ||
        id_ex.instr.opcode == "MUL" || id_ex.instr.opcode == "AND" ||
        id_ex.instr.opcode == "OR"  || id_ex.instr.opcode == "SLL" ||
        id_ex.instr.opcode == "SRL") {
        ex_mem.destReg = id_ex.instr.rd;
    } else {
        ex_mem.destReg = id_ex.instr.rt;
    }

    if (id_ex.ctrl.Branch && ex_mem.aluResult == 0)
        PC = id_ex.pc + 1 + id_ex.instr.immediate;

    if (id_ex.ctrl.Jump)
        PC = id_ex.instr.address;
}
```

Memory Application

- Passes the instruction, control signals, ALU result, and destination register from the EX/MEM latch into the MEM/WB latch
- If MemWrite is set (SW instruction), writes the rt register value into memory using the ALU result as the address
- If MemWrite is set (SW instruction), writes the rt register value into memory using the ALU result as the address
- If the latch is empty (bubble), it clears the MEM/WB latch and returns early so nothing happens
- The memory data read by LW gets stored in the MEM/WB latch where the write back stage will pick it up and write it to a register

```
void memAccess() {
    if (ex_mem.instr.opcode.empty()) { mem_wb = MEM_WB(); return; }

    mem_wb.instr     = ex_mem.instr;
    mem_wb.ctrl      = ex_mem.ctrl;
    mem_wb.aluResult = ex_mem.aluResult;
    mem_wb.destReg   = ex_mem.destReg;

    if (ex_mem.ctrl.MemRead)
        mem_wb.memData = memory[ex_mem.aluResult];

    if (ex_mem.ctrl.MemWrite)
        memory[ex_mem.aluResult] = ex_mem.writeData;
}
```

Write Back

- Final stage of the pipeline
- Writes results back to register file
  - Controlled by RegWrite
- Data source
  - ALU result (R-type, arithmetic, logic)
  - Memory data (LW instructions)
- **MemtoReg Decision**
  - Signal determines the source of the data (0 or 1)
- Destination register
  - Result is given here

```
//Write Back

void writeBack() {
    if (mem_wb.instr.opcode.empty()) return;

    if (mem_wb.ctrl.RegWrite && mem_wb.destReg != 0) {
        if (mem_wb.ctrl.MemToReg)
            registers[mem_wb.destReg] = mem_wb.memData;
        else
            registers[mem_wb.destReg] = mem_wb.aluResult;
    }
}
```

Any questions?
Ask away!

Thank you